on a wide range of web frameworks from various languages for several years now. In round 20 of the plain-text benchmarks, TechEmpower announced that ASP.NET Core with Kestrel was among the 10 fastest of more than 400 frameworks tested![3]

Web servers: Naming things is hard

One difficult aspect of programming for the web is the confusing array of often-conflicting terminology. If you've used IIS, for example, you may have described it as a web server or possibly a web host. Conversely, if you've ever built an application with Node.js, you may have also referred to that application as a web server. Or you may have called the physical machine on which your application runs a web server. Similarly, you may have built an application for the internet and called it a website or a web application, probably somewhat arbitrarily based on the level of dynamism it displayed.

In this book, when I say *web server* in the context of ASP.NET Core, I'm referring to the HTTP server that runs as part of your ASP.NET Core application. By default, this server is the Kestrel web server, but that's not a requirement. It's possible to write a replacement web server for Kestrel if you so desire.

The web server is responsible for receiving HTTP requests and generating responses. In the previous version of ASP.NET, IIS took this role, but in ASP.NET Core, Kestrel is the web server.

I'll use the term *web application* in this book to describe ASP.NET Core applications, regardless of whether they contain only static content or are dynamic. Either way, these applications are accessed via the web, so that name seems to be the most appropriate.

Many of the performance improvements made to Kestrel came not from the ASP.NET team members themselves, but from contributors to the open-source project on GitHub (https://github.com/dotnet/aspnetcore). Developing in the open means that you typically see fixes and features make their way to production faster than you would for the previous version of ASP.NET,